Python: 180 Questions & Answers

Beginner to Intermediate Interview / Viva Preparation

Coverage: Python basics, execution model, data types, operators, conditionals, loops, lists, tuples, memory management, and eval().

1. What is Python?

Answer: Python is a high-level, general-purpose programming language known for readable syntax and a large standard library. It supports procedural, object-oriented, functional, and other programming styles.

2. What are the key features of Python?

Answer: Python has simple syntax, dynamic typing, automatic memory management, portability, a large standard library, a huge ecosystem, exception handling, and support for multiple programming paradigms.

3. Why is Python called an interpreted language?

Answer: Python is commonly called interpreted because Python source code is executed by the Python runtime rather than being directly compiled by the programmer into native machine code. In CPython, source is first compiled to bytecode and that bytecode is executed by the Python Virtual Machine.

4. What is the difference between compiled and interpreted languages?

Answer: A compiled language is generally translated into machine code before execution, while an interpreted language is generally executed by a runtime/interpreter. In practice, modern languages often use a mixture of compilation and interpretation, so the distinction is not absolute.

5. Why is Python platform independent?

Answer: Python programs are portable because the same Python source can run on different operating systems as long as a compatible Python implementation is installed. Python abstracts many OS-specific details.

6. What makes Python dynamically typed?

Answer: A variable name does not have a permanently declared type. The object referenced by the name has a type, and the same name can later refer to an object of another type.

7. What is bytecode in Python?

Answer: Bytecode is an intermediate, platform-independent representation of Python instructions. CPython compiles source code into bytecode before executing it in the Python virtual machine.

8. What is PVM (Python Virtual Machine)?

Answer: The Python Virtual Machine is the runtime mechanism that executes Python bytecode. In CPython, it is implemented as part of the Python interpreter.

9. What is a .pyc file?

Answer: A .pyc file contains cached compiled Python bytecode. CPython may create it to avoid recompiling unchanged modules every time they are imported.

10. What is the purpose of the __pycache__ folder?

Answer: It stores cached bytecode files, normally .pyc files, for imported Python modules. This can make later imports faster.

11. How does Python execute a program internally?

Answer: Conceptually: source code is tokenized and parsed, an AST is produced, the code is compiled into bytecode, and the Python runtime executes that bytecode. CPython may cache bytecode for modules in __pycache__.

12. What is source code?

Answer: Source code is the human-readable program written in a programming language such as Python. Example: print('Hello').

13. What is machine code?

Answer: Machine code is the low-level binary instruction format directly understood by a particular CPU architecture.

14. What is compilation in Python?

Answer: Compilation in Python usually means converting Python source into an intermediate code object/bytecode representation. It does not normally mean compiling the whole program directly into native CPU machine code.

15. What is parsing in Python?

Answer: Parsing is the process of analyzing Python tokens according to Python's grammar to determine the program's syntactic structure.

16. What is a parse tree?

Answer: A parse tree is a tree representation of how source code matches the grammar of a language. It contains grammar-oriented details and can be more detailed than needed for later processing.

17. What is an Abstract Syntax Tree (AST)?

Answer: An AST is a structured tree representing the meaningful syntactic constructs of a program, such as expressions, assignments, calls, and control statements, while omitting many grammar details such as punctuation.

18. Difference between parse tree and AST?

Answer: A parse tree closely reflects the grammar and usually contains more syntactic detail. An AST is a simplified semantic/syntactic structure that keeps the constructs needed for later compiler stages.

19. What role does AST play in execution?

Answer: The AST represents the program structure after parsing and can be inspected or transformed by tools. In CPython, the compiler uses the parsed structure to produce a code object/bytecode that the runtime executes.

20. Is Python fully interpreted? Explain.

Answer: No. The phrase 'fully interpreted' is an oversimplification for CPython. Python source is compiled into bytecode and that bytecode is executed by the Python runtime. Other Python implementations may use different execution strategies.

21. What is a variable?

Answer: A variable is a name that refers to an object. For example, x = 10 binds the name x to an integer object.

22. Why is there no need to declare variable types?

Answer: Python uses dynamic typing, so the type is associated with the object rather than being declared in advance for the variable name.

23. What is dynamic typing?

Answer: Dynamic typing means type information is determined at runtime and a name can refer to objects of different types at different times.

24. Can a variable change its type?

Answer: Yes. For example, x = 10 followed by x = 'hello' makes x refer first to an integer and then to a string.

25. What are built-in data types in Python?

Answer: Common built-in types include int, float, complex, bool, str, list, tuple, set, frozenset, dict, range, bytes, bytearray, memoryview, and NoneType.

26. What is the None type?

Answer: None is the single value of the NoneType. It commonly represents 'no value' or the absence of a meaningful result. Example: result = None.

27. What is mutable and immutable?

Answer: A mutable object can be changed after creation, such as a list or dictionary. An immutable object cannot be changed after creation, such as int, str, tuple, and frozenset.

28. Why are strings immutable?

Answer: Strings are immutable because their contents cannot be changed in place. This provides predictable behavior, allows safe sharing/interning in some cases, and lets strings be hashable so they can be dictionary keys.

29. What is type() function?

Answer: type(obj) returns the type/class of an object. Example: type(10) returns <class 'int'>.

30. What is isinstance()?

Answer: isinstance(obj, cls) checks whether an object is an instance of a class or one of its subclasses. Example: isinstance(10, int) returns True.

31. What is an identifier?

Answer: An identifier is a name used to identify variables, functions, classes, modules, and other program elements.

32. What are the rules for naming identifiers?

Answer: An identifier may contain letters, digits, and underscores; it cannot start with a digit; it cannot be a Python keyword; and Python identifiers are case-sensitive.

33. Can an identifier start with a number?

Answer: No. 2name is invalid. A name such as name2 is valid.

34. What is the difference between identifier and keyword?

Answer: An identifier is a programmer-defined name. A keyword is a reserved word with a special meaning in Python, such as if, for, class, and return.

35. Is Python case-sensitive?

Answer: Yes. name, Name, and NAME are different identifiers.

36. What is multiple assignment?

Answer: Multiple assignment assigns values to multiple names in one statement, for example: a, b = 10, 20.

37. What is chain assignment?

Answer: Chain assignment assigns the same object/value to several names, for example: a = b = c = 0.

38. How do you swap two variables?

Answer: Use tuple unpacking: a, b = b, a. It swaps the references without requiring a temporary variable.

39. What is object identity?

Answer: Object identity indicates whether two references point to the same object. Identity is tested with the is operator.

40. What does id() function do?

Answer: id(obj) returns an integer identifying the object during its lifetime. In CPython it commonly corresponds to the object's memory address, but Python code should not rely on that detail.

41. What is input()?

Answer: input() reads a line from standard input and returns the entered text as a string, without the trailing newline.

42. Why does input() always return a string?

Answer: Because input represents text entered by the user. Python does not automatically assume whether that text should be an integer, float, or something else; you explicitly convert it when needed.

43. What is print()?

Answer: print() writes objects to standard output, usually the console. It can print multiple values and supports options such as sep and end.

44. How do you take integer input?

Answer: Use int(input()). Example: age = int(input('Enter age: ')).

45. How do you take multiple inputs in one line?

Answer: A common method is: a, b = input().split(). For integers: a, b = map(int, input().split()).

46. What are sep and end parameters?

Answer: sep specifies the separator between multiple arguments, while end specifies what is printed after all arguments. Defaults are sep=' ' and end='\n'.

47. What is type casting?

Answer: Type casting/conversion means converting a value from one type to another, such as int('25') or float(10).

48. What is implicit conversion?

Answer: Implicit conversion is automatic type conversion performed by Python in contexts where it is safe or defined. Example: 2 + 3.5 produces 5.5 because the integer is promoted to a float.

49. What is explicit conversion?

Answer: Explicit conversion is conversion requested by the programmer using functions such as int(), float(), str(), and bool().

50. Can every string be converted to int?

Answer: No. A string must represent a valid integer format accepted by int(). int('123') works, but int('12.5') and int('abc') raise ValueError.

51. What happens when float is converted to int?

Answer: int() removes the fractional part by truncating toward zero. int(3.9) is 3 and int(-3.9) is -3.

52. What error occurs when conversion fails?

Answer: A failed numeric conversion commonly raises ValueError. Some invalid argument types can instead produce TypeError.

53. What is int()?

Answer: int() creates an integer from a compatible value or converts a number/string to an integer. Example: int('25') returns 25.

54. What is float()?

Answer: float() creates a floating-point number from a compatible value. Example: float('2.5') returns 2.5.

55. What is str()?

Answer: str() returns a string representation of an object. Example: str(25) returns '25'.

56. What is bool()?

Answer: bool() converts a value to True or False according to Python's truth-value rules.

57. What is string formatting?

Answer: String formatting means inserting values into a string in a controlled way. Common methods are f-strings, str.format(), and older %-formatting.

58. What are f-strings?

Answer: F-strings are formatted string literals prefixed with f. Example: name='Sahil'; print(f'Hello {name}'). Expressions can also be placed inside braces.

59. Difference between format() and f-strings?

Answer: Both support formatted values. f-strings usually offer cleaner syntax when values are already in scope, while format() is useful when a format template is stored or reused.

60. How do you print variables inside strings?

Answer: Use an f-string, for example: name='Sahil'; age=21; print(f'{name} is {age} years old.').

61. What is the difference between / and //?

Answer: / performs true division and normally returns a float. // performs floor division, returning the floor of the quotient.

62. What is the use of % operator?

Answer: % gives the remainder after division. Example: 10 % 3 is 1.

63. What is the use of ** operator?

Answer: ** is the exponentiation operator. Example: 2 ** 3 is 8.

64. Why is ^ not a power operator?

Answer: ^ is the bitwise XOR operator in Python. Exponentiation uses **.

65. What are assignment operators?

Answer: Assignment operators assign or update values. Examples include =, +=, -=, *=, /=, //=, %=, **=, &=, |=, ^=, <<=, and >>=.

66. Difference between = and ==?

Answer: = assigns a value/reference to a name. == compares values for equality.

67. What does += do internally?

Answer: Conceptually, x += y attempts an in-place addition operation when supported; otherwise it can fall back to creating a new result and rebinding x. For immutable objects such as integers, a new object is produced.

68. What are relational operators?

Answer: Relational/comparison operators compare values: <, <=, >, >=, ==, and !=. Their result is normally True or False.

69. Can relational operators be chained?

Answer: Yes. For example, 5 > 3 > 1 means 5 > 3 and 3 > 1. The middle expression is evaluated only once.

70. What is the output of 5 > 3 > 1?

Answer: True.

71. What are logical operators?

Answer: Python's logical operators are and, or, and not. They combine or negate truth values and, importantly, and/or return one of their operands rather than always returning True or False.

72. Difference between and and &?

Answer: and is logical short-circuiting and works with truthiness. & is bitwise AND for integers and is also overloaded by some types such as sets.

73. Difference between or and |?

Answer: or is logical short-circuiting and returns an operand. | is bitwise OR for integers and is also overloaded by types such as sets and dictionaries in supported operations.

74. What does not operator do?

Answer: not reverses the truth value of its operand: not True is False and not 0 is True.

75. What are membership operators?

Answer: in and not in test whether an item belongs to a container or sequence, such as a string, list, tuple, set, or dictionary.

76. Difference between in and not in?

Answer: in returns True when the item is found; not in returns True when it is not found.

77. What are identity operators?

Answer: is and is not compare object identity, meaning whether two references refer to the same object.

78. Difference between is and ==?

Answer: == compares values/equality. is checks whether the two references are the same object. Use == for value comparison and is commonly for singleton checks such as x is None.

79. What are bitwise operators?

Answer: For integers, bitwise operators operate on individual bits: &, |, ^, ~, <<, and >>.

80. What is XOR operator?

Answer: XOR (^) gives 1 where corresponding bits differ and 0 where they are the same. Example: 5 ^ 3 is 6.

81. What is an if statement?

Answer: if executes a block when its condition is truthy. Example: if age >= 18: print('Adult').

82. What is if-else?

Answer: if-else chooses between two blocks: the if block runs when the condition is truthy; otherwise the else block runs.

83. What is elif ladder?

Answer: An elif ladder checks multiple conditions in order. The first truthy condition's block executes; if none is true, the optional else block executes.

84. What is nested if?

Answer: A nested if is an if statement placed inside another conditional block. It is useful when a second condition should be checked only after an outer condition succeeds.

85. What is a ternary operator?

Answer: Python's conditional expression has the form value_if_true if condition else value_if_false. Example: result = 'Pass' if marks >= 40 else 'Fail'.

86. What values are considered False in Python?

Answer: False, None, numeric zero values such as 0, 0.0 and 0j, and empty strings/containers such as '', [], (), {}, set(), and range(0) are falsey.

87. What is truthiness?

Answer: Truthiness is Python's rule for deciding whether an object behaves as true or false in a Boolean context such as if or while.

88. What is match-case?

Answer: match-case, introduced in Python 3.10, provides structural pattern matching. It can match literal values, patterns, sequences, mappings, classes, and more.

89. How is match-case different from switch?

Answer: A traditional switch usually selects among constant cases based on one expression. Python's match-case is more powerful because it supports structural patterns, guards, destructuring, and class patterns.

90. What is wildcard (_) in match-case?

Answer: A standalone underscore acts as a wildcard pattern that matches anything and is commonly used as the final/default case.

91. Can match-case handle multiple values?

Answer: Yes. Use OR patterns with |. Example: case 1 | 2 | 3: matches any of those values.

92. When should match-case be used?

Answer: Use it when pattern matching makes the logic clearer, especially for structured data or many distinct patterns. For simple conditions, if/elif may be clearer.

93. Can match-case match patterns?

Answer: Yes. It can match literals, sequences, mappings, class instances, alternatives, and patterns with guards.

94. What happens when no case matches?

Answer: No case block executes and execution continues after the match statement. There is no automatic error.

95. Can match-case replace if-elif ladders?

Answer: Sometimes, especially for value or structural patterns. It cannot replace every if-elif ladder because arbitrary Boolean conditions are often better expressed with if/elif.

96. What is a for loop?

Answer: A for loop iterates over the items of an iterable, such as a list, string, tuple, set, dictionary, or range.

97. What is a while loop?

Answer: A while loop repeatedly executes its body as long as its condition remains truthy.

98. Difference between for and while?

Answer: for is generally used when iterating over an iterable or known sequence of items. while is used when repetition depends on a condition and the number of iterations may not be known in advance.

99. What is range()?

Answer: range() represents an arithmetic progression of integers and is commonly used for counting loops. It is a lazy range object, not a full list.

100. Explain range(start, stop, step).

Answer: start is the first value, stop is exclusive, and step is the increment/decrement. Example: range(2, 10, 2) produces 2, 4, 6, 8.

101. Can range have negative step?

Answer: Yes. Example: range(5, 0, -1) produces 5, 4, 3, 2, 1.

102. What is an infinite loop?

Answer: An infinite loop is a loop whose condition never becomes false or whose iteration never naturally ends. Example: while True: ...

103. How do you stop an infinite loop?

Answer: Use a correct terminating condition, break when an exit condition is reached, or interrupt the program externally (commonly Ctrl+C in a terminal).

104. What is a nested loop?

Answer: A nested loop is a loop inside another loop. For each iteration of the outer loop, the inner loop normally completes its iterations.

105. What is loop control?

Answer: Loop control means changing the normal flow of a loop using statements such as break, continue, and pass.

106. What is break?

Answer: break immediately terminates the nearest enclosing loop and continues execution after that loop.

107. What is continue?

Answer: continue skips the rest of the current iteration and moves to the next iteration of the nearest enclosing loop.

108. What is pass?

Answer: pass is a do-nothing statement. It is used where Python requires a statement syntactically but you intentionally want no action.

109. Difference between break and continue?

Answer: break exits the loop completely. continue skips only the current iteration and keeps the loop running.

110. What is for-else?

Answer: A for loop can have an else block. The else executes if the loop finishes normally without encountering break.

111. What is while-else?

Answer: A while loop can also have an else block. The else executes when the while condition becomes false normally, not when break terminates the loop.

112. When does loop else execute?

Answer: It executes when the loop completes normally without break. For while, that means the condition becomes false; for for, the iterable is exhausted.

113. Does else execute after break?

Answer: No. If break terminates the loop, the loop's else block is skipped.

114. What is loop nesting complexity?

Answer: If an outer loop runs n times and an inner loop runs n times for each outer iteration, the total work is typically O(n^2). The exact complexity depends on the loop bounds and operations.

115. How do you iterate through a string?

Answer: Use a for loop: for ch in 'Python': print(ch). Each iteration gives the next character.

116. What is a list?

Answer: A list is an ordered, mutable collection that can store multiple objects and can contain duplicates and mixed types.

117. Why are lists mutable?

Answer: A list stores references to its elements and provides operations that can change the collection in place, such as append, remove, and item assignment.

118. How do you create a list?

Answer: Use square brackets, for example: nums = [1, 2, 3]. You can also use list(iterable).

119. What is indexing in lists?

Answer: Indexing accesses an individual element by position. Python uses zero-based indexing, so the first element is at index 0.

120. What is negative indexing?

Answer: Negative indices count from the end. -1 is the last element, -2 is the second last, and so on.

121. What is slicing?

Answer: Slicing extracts a portion of a sequence using [start:stop:step]. The stop index is excluded.

122. Difference between append() and extend()?

Answer: append(x) adds x as one element. extend(iterable) adds each element from the iterable. [1].append([2,3]) gives [1,[2,3]], while extend gives [1,2,3].

123. Difference between append() and insert()?

Answer: append(x) adds to the end. insert(index, x) inserts at a specified position.

124. Difference between remove() and pop()?

Answer: remove(x) deletes the first matching value and raises ValueError if it is absent. pop(index) removes and returns an element by index; without an index it removes the last element.

125. What does clear() do?

Answer: clear() removes all elements from a mutable collection such as a list, leaving an empty collection.

126. What does copy() do?

Answer: For a list, copy() creates a shallow copy of the list. The outer list is new, but nested referenced objects are shared.

127. Difference between sort() and sorted()?

Answer: list.sort() sorts a list in place and returns None. sorted(iterable) returns a new sorted list and leaves the original iterable unchanged.

128. Difference between reverse() and reversed()?

Answer: list.reverse() reverses a list in place and returns None. reversed(iterable) returns an iterator that yields elements in reverse order.

129. How do you find list length?

Answer: Use len(list_name). Example: len([10, 20, 30]) is 3.

130. How do you find max and min values?

Answer: Use max(iterable) and min(iterable). Example: max([3, 8, 2]) is 8 and min(...) is 2.

131. How do you count occurrences?

Answer: Use list.count(value). Example: [1,2,1,1].count(1) returns 3.

132. How do you merge two lists?

Answer: Use concatenation, a+b, or unpacking such as [*a, *b]. extend() can also add one list's elements to another in place.

133. How do you remove duplicates?

Answer: For hashable items, set(list) removes duplicates but does not preserve the original order in the general set abstraction. To preserve first-seen order, use dict.fromkeys(list) or an explicit loop.

134. What is a nested list?

Answer: A nested list is a list containing one or more lists, such as [[1,2], [3,4]].

135. How do you traverse a nested list?

Answer: Use nested loops. Example: for row in matrix: for value in row: print(value).

136. What is shallow copy?

Answer: A shallow copy creates a new outer container but keeps references to the same nested objects.

137. What is deep copy?

Answer: A deep copy recursively copies nested objects so the copied structure does not share mutable nested objects with the original, where the copy operation supports copying them.

138. Difference between shallow and deep copy?

Answer: Shallow copy duplicates only the outer container; deep copy recursively duplicates nested objects. Therefore, changes to shared nested mutable objects can affect both structures after a shallow copy.

139. What happens in a=b?

Answer: No copy is made. Both names refer to the same object, so mutations through either name affect that same mutable object.

140. What happens in a=b.copy()?

Answer: For a list, a new outer list is created. The elements are copied by reference, so nested mutable objects remain shared.

141. What is a tuple?

Answer: A tuple is an ordered, immutable collection. It is written using commas, commonly with parentheses, for example (1, 2, 3).

142. Why are tuples immutable?

Answer: Once a tuple is created, its element references cannot be changed, added, or removed. This makes tuples suitable for fixed collections and allows them to be hashable when all their elements are hashable.

143. How do you create a tuple?

Answer: Use comma-separated values, commonly with parentheses: t = (1, 2, 3). tuple(iterable) can also create a tuple.

144. What is a single-element tuple?

Answer: A one-element tuple requires a trailing comma: (5,). The comma, not the parentheses, makes it a tuple.

145. Difference between (5) and (5,)?

Answer: (5) is simply the integer 5 because parentheses group an expression. (5,) is a tuple containing one element.

146. What is tuple packing?

Answer: Tuple packing means placing multiple values into one tuple, for example: t = 10, 20, 30.

147. What is tuple unpacking?

Answer: Tuple unpacking assigns elements of an iterable to multiple names, for example: a, b = (10, 20).

148. What is extended unpacking?

Answer: Extended unpacking uses * to collect multiple remaining items. Example: a, *middle, b = [1,2,3,4] gives a=1, middle=[2,3], b=4.

149. What is a nested tuple?

Answer: A nested tuple contains another tuple as an element, for example: ((1,2), (3,4)).

150. Can a tuple contain mutable objects?

Answer: Yes. A tuple itself is immutable, but it can contain a mutable object such as a list. That list can still be changed.

151. Advantages of tuple over list?

Answer: Tuples are immutable, can communicate fixed data, can be hashable when all elements are hashable, and often use slightly less memory than equivalent lists.

152. Which is faster: list or tuple?

Answer: For many fixed-size read-only operations, tuples can be slightly faster than lists because they are simpler immutable containers. The difference is workload-dependent.

153. Which consumes less memory?

Answer: A tuple generally consumes less memory than a list holding the same number of references, though exact memory usage depends on the implementation and contents.

154. When should tuple be preferred?

Answer: Prefer a tuple for fixed collections of values, records that should not be changed, or data that may need to be used as a dictionary key when all elements are hashable.

155. Can a tuple be a dictionary key?

Answer: Yes, if the tuple and all objects inside it are hashable. A tuple containing a list cannot be a dictionary key.

156. What is the output of print(True + True)?

Answer: 2. bool is a subclass of int, so True behaves numerically as 1 in arithmetic.

157. What is the output of print(False + True)?

Answer: 1.

158. What is the output of print(bool(''))?

Answer: False, because an empty string is falsey.

159. What is the output of print(bool('0'))?

Answer: True, because the string '0' is non-empty. Its text content does not make it numerically zero.

160. What is the output of print(10 and 0 or 5)?

Answer: 5. and returns the first falsey operand (0); then 0 or 5 returns 5.

161. What is the output of print(5 > 3 > 2)?

Answer: True, because both comparisons 5 > 3 and 3 > 2 are true.

162. What is the output of print(10/3)?

Answer: 3.3333333333333335 in typical Python 3 floating-point output.

163. What is the output of print(10//3)?

Answer: 3.

164. What is the output of print(10%3)?

Answer: 1.

165. What is the output of print(2**3**2)?

Answer: 512. Exponentiation is right-associative, so it is evaluated as 2 ** (3 ** 2) = 2 ** 9.

166. Why is Python slower than Java?

Answer: CPython commonly executes bytecode through an interpreter/runtime and has dynamic type checks and object overhead. Java commonly uses JVM JIT compilation to optimize frequently executed code into native machine instructions. The actual performance depends on the implementation and workload.

167. Why is Python popular in AI?

Answer: Python has a simple syntax, rapid development cycle, and a rich ecosystem including NumPy, pandas, scikit-learn, PyTorch, TensorFlow, Jupyter, and many scientific libraries.

168. Why does Python support multiple paradigms?

Answer: Python's design provides functions, classes, first-class objects, higher-order functions, generators, and flexible syntax, allowing procedural, object-oriented, functional, and other styles.

169. What happens when a syntax error occurs?

Answer: Python cannot parse the invalid source, so execution of that code does not begin. It reports a SyntaxError (or a related parsing error) with location information when possible.

170. What happens when an exception is not handled?

Answer: The exception propagates up the call stack. If no matching handler is found, Python terminates the program and prints a traceback describing the exception.

171. Why are lists mutable but tuples immutable?

Answer: They serve different purposes. Lists are designed for collections that may change, while tuples represent fixed ordered collections. Their internal implementations and APIs enforce these behaviors.

172. Why does input() return string?

Answer: Because keyboard input arrives as text. Automatic conversion could be ambiguous, so Python leaves type interpretation to the programmer.

173. What happens when Python starts execution?

Answer: Python initializes its runtime, loads required modules/environment information, parses and compiles the requested source, and begins executing the resulting code. For a script, top-level statements execute from top to bottom.

174. How does memory management work in Python?

Answer: Python manages objects automatically. In CPython, memory management primarily uses reference counting plus a cyclic garbage collector, with specialized allocators for Python objects.

175. What is garbage collection in Python?

Answer: Garbage collection is automatic reclamation of memory that is no longer needed. CPython's cyclic garbage collector detects unreachable reference cycles that reference counting alone cannot reclaim.

176. difference list versus tuple versus set

Answer: List: ordered, mutable, allows duplicates, indexed. Tuple: ordered, immutable, allows duplicates, indexed. Set: mutable collection of unique hashable elements, optimized for membership tests, and not indexable.

177. what is empty set

Answer: An empty set is created with set(), not {}. {} creates an empty dictionary. Example: s = set().

178. can we use sort() with tuple why or why not.

Answer: No, a tuple has no sort() method because it is immutable. Use sorted(tuple), which returns a new list. Example: sorted((3,1,2)) returns [1,2,3].

179. what is tuple comprehension?? explain in detail.

Answer: Python does not have a true tuple-comprehension syntax. An expression such as (x*x for x in range(5)) is a generator expression, not a tuple. To create a tuple from it, use tuple(x*x for x in range(5)). A list comprehension uses [x*x for x in range(5)] and immediately creates a list, whereas the generator expression produces values lazily.

180. what is eval()?? explain with proper examples.

Answer: eval(expression) evaluates a Python expression supplied as a string and returns its result. Example: eval('2 + 3') returns 5; x=10; eval('x * 2') returns 20. It can evaluate expressions involving variables available in its namespace, but it does not execute statements such as for-loops or assignments in the normal way. Security warning: never use eval() on untrusted user input because it can execute dangerous Python expressions and access sensitive resources. For simple data formats, prefer safer alternatives such as json.loads().